Let's move on to the public API, that will mirror the API for `set` that we defined in section 4.3.3, and let’s start, obviously, from the constructor method.

As it happens most of the times, the task for our constructor is mostly boilerplate code to set up all the internal state of a Bloom filter; in this case, however, there is also some non-trivial math to work out in order to compute the resources to allocate in order for the container to live up to the accuracy required by the client.

Listing 4.9 describes the code for a possible constructor: notice, in particular, at lines #5, and #8, how we compute respectively the number of bits and the number of hash functions needed to have a ratio of false positives within the tolerance specified by `maxTolerance` (and consequently, at line #9, the number of array elements needed to store the filter). Here we assume that we use an array whose elements are *integers*, and `BITS_PER_INT` is a system variable that gives us the size, in bits, of integers; clearly, for those languages supporting multiple numerical types, we can also choose to have arrays of *bytes*, when available.

**Listing 4.9 Bloom filter’s constructor**

```
function BloomFilter(maxSize, maxTolerance=0.01,
seed=random())               #1
  this.size ←
0
#2
  this.maxSize ←
maxSize
#3
```